

算法综述（适用范围 / 复杂度 / 用法）

- 用途：做题时先查这份，不查自己想。
- 三条查法：① 这题要做什么 → 查「按问题查解法」；② 这个结构能干什么、代价多少 → 查「结构总表」；③ 数据量给了 → 查「复杂度预算表」。
- 维护规则：每学完一个专题，回来补一行，而不是另开一个文件。行里必须有「一句话用法」和「笔记链接」。

一、按「问题」查解法（适用范围表）

你想做的事	首选	备选	复杂度
静态区间和	前缀和	ST 表? $\times$ （不满足可重复贡献）	$O(1)$
单点修改 + 区间和	树状数组	线段树	$O(\log n)$
区间修改 + 区间和	线段树 + lazy	差分（离线/单次）	$O(\log n)$
静态区间最值 / gcd	ST 表	线段树	$O(1)$ 查询, $O(n \log n)$ 预处理
动态区间最值	线段树	—	$O(\log n)$
滑动窗口最值	单调队列	multiset	$O(n)$ （比 multiset 的 $O(n \log n)$ 快）
第 k 小（静态、一次）	nth_element	堆	平均 $O(n)$
第 k 小（带修改）	树套树 / 整体二分	主席树	$O(\log^2 n)$
单串匹配	KMP	哈希	$O(n+m)$
多模式串匹配	AC 自动机	逐个 KMP	$O(\text{总长})$
前缀统计 / 字典序	字典树	排序	$O(\text{len})$ 单次
异或最优（一对）	01-Trie	—	$O(\text{位数})$
子集异或最大值	线性基	—	$O(\text{位数})$
动态加边连通性	并查集	DFS	均摊近 $O(1)$
动态删边连通性	可撤销并查集 + 线段树分治	LCT	$O(\log n)$
单源最短路（非负权）	Dijkstra + 堆	—	$O(m \log n)$
单源最短路（有负边）	Bellman-Ford / SPFA	—	$O(nm)$
全源最短路	Floyd	n 次 Dijkstra	$O(n^3)$
最小生成树	Kruskal	Prim（稠密图）	$O(m \log m)$
拓扑序 / 判环	Kahn（BFS 入度）	DFS 三色	$O(n+m)$
强连通分量	Tarjan	Kosaraju	$O(n+m)$
LCA	倍增 / 树剖	欧拉序 + ST 表	$O(\log n)$ / $O(1)$
区间第 k 小（不带修改）	主席树	整体二分	$O(\log n)$
$n \leq 20$ 的集合选择	状压 dp	暴力枚举	$O(2^n \cdot n)$

你想做的事	首选	备选	复杂度
$n \leq 40$ 的选一半	meet in the middle	—	$O(2^{(n/2)})$

二、结构总表（能干什么 / 代价 / 怎么用）

结构	适用场景	建表	单次操作	空间	一句话用法	笔记
前缀和 / 差分	静态区间和 / 区间加	$O(n)$	$O(1)$	$O(n)$	$s[i]=s[i-1]+a[i]$ ；差分 $d[l]+=x, d[r+1]-=x$	
双指针 / 滑动窗口	单调性区间	—	$O(n)$ 摊还	$O(1)$	右端扩展 + 左端收缩	
二分答案	单调可判定	—	$O(\log V \cdot \text{check})$	$O(1)$	二分值域， $\text{check}(\text{mid})$ 判可行性	
单调栈	下一个更大 / 直方图面积	$O(n)$	$O(1)$ 摊还	$O(n)$	栈内维持单调， 弹出时计算贡献	 Monotonic_Stack/
单调队列	滑动窗口最值	$O(n)$	$O(1)$ 摊还	$O(n)$	队首过期弹出， 队尾破坏单调弹出	 Monotonic_Queue/
并查集	连通性 / 集合合并	$O(n)$	均摊近 $O(1)$	$O(n)$	find 路径压缩 + union 按秩	 union-find_sets/
ST 表	静态区间最值 / gcd	$O(n \log n)$	$O(1)$	$O(n \log n)$	两块 2^k 覆盖，lg[] 表	 Sparse Table/st.md
树状数组	前缀和 + 单点改	$O(n)$	$O(\log n)$	$O(n)$	lowbit(x)=x&-x；倒序/ 正序枚举	 Binary Indexed Tree/
线段树	区间合并类信息	$O(n)$	$O(\log n)$	$O(4n)$	递归/迭代；lazy 标记下推	 Segment_Tree/
字典树	前缀统计、字符串集合	$O(\text{总长})$	$O(\text{len})$	$O(\text{节点数} \times K)$	tree[cur][path]，根为 0	 Trie/字典树.md
散列表（哈希）	精确匹配、计数	$O(n)$	期望 $O(1)$	$O(n)$	单哈希易被卡，双哈希 + 随机 base	 Character_Hash_Array/
堆 / priority_queue	动态最值、贪心	$O(n)$	$O(\log n)$	$O(n)$	默认大顶堆，小顶堆用 greater<> 或取负	
单调队列优化的 dp	转移带窗口最值	$O(n)$	$O(1)$ 摊还	$O(n)$	把 dp 转移改写成窗口形式	
主席树	带历史的区间查询	$O(n \log n)$	$O(\log n)$	$O(n \log n)$	每个版本只新建 $\log n$ 个节点	
整体二分	一堆询问一起二分	—	$O((n+q) \log^2)$	$O(n+q)$	值域二分 + 询问分治	
线性基	子集异或	$O(n \cdot B)$	$O(B)$	$O(B)$	高斯消元在异或域上的形式	
AC 自动机	多模式匹配	$O(\text{总长})$	$O(\text{len})$	$O(\text{节点数} \times K)$	Trie + fail 指针 (fail = 最长真后缀)	

结构	适用场景	建表	单次操作	空间	一句话用法	笔记
KMP	单模式匹配	$O(m)$	$O(n)$	$O(m)$	<code>next[]</code> = 最长相同前后缀	
Manacher	最长回文子串	$O(n)$	—	$O(n)$	插分隔符 + 回文半径	
树剖 / 倍增 LCA	树上路径	$O(n \log n)$	$O(\log n)$	$O(n \log n)$	dfn 序把路径拆成 $O(\log n)$ 段	

= 有笔记文件； = 笔记待建。每补一个专题，把 改成 并填上链接。

最小可复制代码块（只给已学过的结构）

只覆盖上表标 的结构。没学过的模板贴上去也没用——模板只有在“你能解释每一行为什么这么写”的时候才叫模板，否则就是复制粘贴。没学的部分本节末尾只给「关键思想」。

前缀和 / 差分

```
long long s[N]; // ⚠️ 前缀和一定用 long long
for (int i = 1; i <= n; i++) s[i] = s[i-1] + a[i];
long long sum = s[r] - s[l-1]; // 区间和 [l, r]

long long d[N + 2]; // 差分数组要开 n+2
void add(int l, int r, long long x) { d[l] += x; d[r+1] -= x; } // 区间 [l,r] += x
for (int i = 1; i <= n; i++) d[i] += d[i-1]; // 前缀和还原
```

- 要点： `s[l-1]` 不是 `s[l]`；差分的威力是“区间修改变成两次单点修改”，离线最后一次性还原。
- ⚠️ 易错：类型（前缀和没开 `long long`）· 边界（`d[r+1]` 越界，数组要开 `n+2`）

双指针 / 滑动窗口

```
int l = 1; long long sum = 0; int ans = 0;
for (int r = 1; r <= n; r++) {
    sum += a[r]; // 右端点进窗
    while (l <= r && !ok(sum)) { // 不满足条件就收缩
        sum -= a[l]; l++;
    }
    ans = max(ans, r - l + 1);
}
```

- 要点： `r` 只前进、`l` 也只前进 → 总移动 $O(n)$ ，不是 $O(n^2)$ 。
- ⚠️ 易错：逻辑（收缩条件写成 `if` 而不是 `while`；`ok` 里忘了 `l <= r`）

二分答案

```
int lo = 1, hi = n; // 答案的取值区间
while (lo < hi) { // 循环不变量：答案在 [lo, hi] 内
    int mid = lo + (hi - lo) / 2; // ⚠️ 防溢出写法
    if (check(mid)) hi = mid; // mid 可行 → 答案 ≤ mid
    else lo = mid + 1; // mid 不可行 → 答案 > mid
}
cout << lo;
```

- 要点： `check(x)` 必须单调（`x` 可行 $\Rightarrow$ 比 `x` 更“宽松”的都可行）；这个写法求“最小可行值”。
- ⚠️ 易错：逻辑（`hi = mid - 1` 与 `hi = mid` 混用导致漏解/死循环）· 边界（答案可能不在 `[1, n]`，比如是 0）

单调栈（下一个更大元素）

```
int st[N], top = 0;
for (int i = 1; i <= n; i++) {
    while (top && a[st[top]] < a[i]) { // 栈里比 a[i] 小的全弹掉
        nxt[st[top--]] = i;           // 弹出时刻 = 它右边第一个更大值的位置
    }
    st[++top] = i;
}
// 直方图最大矩形：弹出时面积 = a[st[top]] * (i - st[top-1] - 1)
```

- **要点：弹出时计算贡献** —— 这是所有单调栈题的统一套路。
-  **易错：** 边界 （栈空时 `st[top-1]`、`st[top]` 越界；哨兵/特判）

单调队列（滑动窗口最值）

```
int q[N], head = 1, tail = 0;
for (int i = 1; i <= n; i++) {
    while (head <= tail && q[head] < i - k + 1) head++; // 队首过期
    while (head <= tail && a[q[tail]] <= a[i]) tail--; // 队尾破坏单调
    q[++tail] = i;
    if (i >= k) ans[i] = a[q[head]]; // 队首就是窗口最值
}
```

- **要点：两步：** 队首过期、队尾破坏单调。 `<=` 还是 `<` 决定重复元素怎么处理。
-  **易错：** 边界 （`k = 1` / `k = n`；`q[head]` 越界）· 实现 （两个 `while` 的顺序）

并查集

```
int fa[N], sz[N];
int find(int x) { // 路径压缩（不写递归会爆栈）
    while (fa[x] != x) { fa[x] = fa[fa[x]]; x = fa[x]; }
    return x;
}
void merge(int a, int b) {
    a = find(a); b = find(b);
    if (a == b) return;
    if (sz[a] < sz[b]) swap(a, b); // 按大小合并
    fa[b] = a; sz[a] += sz[b];
}
// 初始化：for (i) fa[i] = i, sz[i] = 1;
```

- **要点：** 路径压缩 + 按秩合并 → 单次均摊近 $O(\alpha(n))$ ；**只有路径压缩是 $O(\log n)$ 摊还，也够用。**
-  **易错：** 初始化 （多测忘了重设 `fa[i] = i`）· 逻辑 （`merge` 里忘了先 `find`）

ST 表（静态区间最值）

```
int f[N][20], lg[N];
lg[0] = -1;
for (int i = 1; i <= n; i++) { cin >> f[i][0]; lg[i] = lg[i >> 1] + 1; }
for (int j = 1; j <= lg[n]; j++)
    for (int i = 1; i + (1 << j) - 1 <= n; i++)
        f[i][j] = max(f[i][j-1], f[i + (1 << (j-1))][j-1]);
int query(int l, int r) {
    int k = lg[r - l + 1];
    return max(f[l][k], f[r - (1 << k) + 1][k]);
}
```

- **要点：**信息必须满足**可重复贡献**（ $op(x,x)=x$ ）→ 最值、gcd 可以，区间和不行。
- **⚠ 易错：**边界（j 上界用 $\lg[n]$ ；查询右半块起点是 $r-2^k+1$ ）· 逻辑（用 $\log_2()$ 浮点导致精度问题）

树状数组（单点修改 + 前缀查询）

```
int t[N];
inline int lowbit(int x) { return x & -x; }
void modify(int i, int x) { for (; i <= n; i += lowbit(i)) t[i] += x; }
int query(int i) { int s = 0; for (; i > 0; i -= lowbit(i)) s += t[i]; return s; }
// 区间和 [l, r] = query(r) - query(l-1)
```

- **要点：** $i += \text{lowbit}(i)$ 向上更新、 $i -= \text{lowbit}(i)$ 向前查询。只有**"可减"信息能用**（和 / 异或可以，最值不行）。
- **⚠ 易错：**初始化（建树是 n 次 `modify`，不是直接 $t[i]=a[i]$ ）· 边界（下标从 1 开始， $i > 0$ 不是 $i \geq 0$ ）

线段树（区间加 + 区间和，带 lazy）

```
long long sum[N << 2], lz[N << 2];
void pushup(int p) { sum[p] = sum[p<<1] + sum[p<<1|1]; }
void apply(int p, int l, int r, long long v) { sum[p] += v * (r - l + 1); lz[p] += v; }
void pushdown(int p, int l, int r) {
    if (!lz[p]) return;
    int m = (l + r) >> 1;
    apply(p<<1, l, m, lz[p]); apply(p<<1|1, m+1, r, lz[p]);
    lz[p] = 0;
}
void modify(int p, int l, int r, int ql, int qr, long long v) {
    if (ql <= l && r <= qr) { apply(p, l, r, v); return; }
    pushdown(p, l, r);
    int m = (l + r) >> 1;
    if (ql <= m) modify(p<<1, l, m, ql, qr, v);
    if (qr > m) modify(p<<1|1, m+1, r, ql, qr, v);
    pushup(p);
}
long long querySum(int p, int l, int r, int ql, int qr) {
    if (ql <= l && r <= qr) return sum[p];
    pushdown(p, l, r);
    int m = (l + r) >> 1; long long res = 0;
    if (ql <= m) res += querySum(p<<1, l, m, ql, qr);
    if (qr > m) res += querySum(p<<1|1, m+1, r, ql, qr);
    return res;
}
```

- **要点：**空间开 $4n$ ；pushdown 只在需要往下走时调用（别忘了 pushup）。
-  **易错：**边界（ $N < 2$ 开少了）· 逻辑（忘记 pushup / pushdown 顺序颠倒）· 类型（sum 用 long long）

字典树（小写字母版，字符集 26）

```
int ch[N][26], cnt[N], idx;           // ch[cur][c] = 子节点编号, 0 表示无边
void insert(const string &s) {
    int cur = 0;
    for (char c : s) {
        int p = c - 'a';
        if (!ch[cur][p]) ch[cur][p] = ++idx;
        cur = ch[cur][p];
    }
    cnt[cur]++;                       // 这里用 cnt 代替 e[]; 要前缀计数再加一个 p[]
}
bool find(const string &s) {
    int cur = 0;
    for (char c : s) {
        int p = c - 'a';
        if (!ch[cur][p]) return false;
        cur = ch[cur][p];
    }
    return cnt[cur] > 0;
}
```

- **要点：**根固定为 0（所以 0 能表示“边不存在”）；**数组大小 = 总字符数 × 字符集**，先估内存（见 Trie/字典树.md）。
-  **易错：**边界（数组按“串数”开）· 时间（多测 memset 整张表）· 初始化（多测只清 0..idx）

字符串哈希

```
typedef unsigned long long ull;
const ull B = 131;                     // base 用质数/随机，别用固定小值
ull h[N], pw[N];
pw[0] = 1;
for (int i = 1; i <= n; i++) pw[i] = pw[i-1] * B;
for (int i = 1; i <= n; i++) h[i] = h[i-1] * B + s[i];
ull get(int l, int r) { return h[r] - h[l-1] * pw[r - l + 1]; } // 子串哈希
```

- **要点：**unsigned long long 自然溢出 = mod 2^{64} ；**需要绝对安全就双哈希**（两个 base/模数）。
-  **易错：**类型（忘了 ull 自然溢出或用了 int）· 逻辑（get 里忘了乘 pw[len]）

堆（priority_queue）

```
priority_queue<int> q;                  // 大顶堆（默认）
priority_queue<int, vector<int>, greater<int>> q2; // 小顶堆
q.push(x); q.top(); q.pop(); q.empty();
// 自定义: priority_queue<Node> 需要 Node::operator<, 且方向与直觉相反
// 清空: priority_queue 没有 clear() — while (!q.empty()) q.pop();
```

- **要点：**只暴露堆顶，没有 erase / 迭代器。要删中间元素就换 set。
-  **易错：**逻辑（operator< 写反 → 大小顶堆搞错）

单调队列优化 DP（窗口最值转移）

```
// 框架: dp[i] = min/max{ dp[j] + cost(j) }, j ∈ [i-k, i-1]
int q[N], head = 1, tail = 0;
for (int i = 1; i <= n; i++) {
    while (head <= tail && q[head] < i - k) head++;           // 队首过期
    dp[i] = dp[q[head]] + cost(i);                           // 用队首转移
    while (head <= tail && dp[q[tail]] >= dp[i]) tail--;      // 队尾破坏单调
    q[++tail] = i;
}
```

- **要点：**把 dp 转移看成**"在滑动窗口里取最值"**，就能把 O(nk) 降到 O(n)。多重背包的单调队列优化是它的变体（下标按 % 体积 分组）。
-  **易错：**逻辑（队首过期的位置算错；把 dp[i] 放进队列的**时机**弄错）

还没学的结构，先记「关键思想」

结构	一句话思想（学到时再补代码）
主席树	每个前缀开一个版本， 每次只新建 log n 个节点 ，查询时两版本相减
整体二分	一堆询问 一起 对值域二分，按"答案在左半/右半"分治到两组
线性基	高斯消元搬到异或域上，维护一组"最高位互不相同"的基向量
AC 自动机	字典树 + fail 指针（= 最长真后缀所在节点），匹配失败时跳 fail
KMP	nxt[i] = 前 i 个字符的 最长相同前后缀 ；失配时跳 nxt 而不是回 0
Manacher	插分隔符统一奇偶长度，维护当前最右回文边界加速扩展
树剖 / 倍增 LCA	dfn 序把树上路径拆成 O(log n) 段连续区间，交给线段树

三、常用 STL 容器：特点 + 操作复杂度

要"用对"容器，先搞清四组性质：**有序 / 无序、能不能随机访问、迭代器什么时候失效、常数大不大**。

3.1 四组性质（先建立直觉）

① 有序 vs 无序

有序（遍历出来就是排序结果）	无序
set / map / multiset / multimap（红黑树，自动排序）	unordered_set / unordered_map（哈希，顺序不可预测）
也能变有序：vector 手动 sort；priority_queue 只保证堆顶 ，内部无序	stack / queue / list：按插入顺序，不算"有序容器"

这一条决定三件事：能不能找前驱/后继（lower_bound）、遍历顺序是否可预测、最坏复杂度有没有保证。
map / set 能 lower_bound；unordered_* **不能**——因为它无序。

② 能否随机访问（O(1) 下标）

能	不能
vector / deque / array / string / bitset	list / set / map / unordered_* / stack / queue / priority_queue

这一条决定 `std::lower_bound` 能不能用：在不能随机访问的容器上用它 = $O(n)$ ，必须换成成员函数。

③ 迭代器 / 引用什么时候失效

只失效被删的那个（节点式）	可能大面积失效（连续式）
list / set / map / multiset / multimap	vector（扩容时全部失效）、deque（插入使迭代器失效）、unordered_*（rehash 时全部失效）

这一条决定“边遍历边删除”能不能写：vector 里 erase 之后旧迭代器就废了，必须 `it = v.erase(it)`。

④ 常数大小（同样 $O(\log n)$ ，谁更快）

vector / array（连续内存，最快）> deque > bitset > 红黑树 set / map > list（每次跳指针，最慢）
哈希 unordered_* 平均比红黑树快，但算哈希本身有开销，且最坏可以被构造数据卡死。

3.2 操作复杂度总表

容器	底层	随机访问	查找	插入	删除	关键特点 / 坑
vector	连续数组	$O(1)$	$O(n)$	尾均摊 $O(1)$ ，中间 $O(n)$	尾 $O(1)$ ，中间 $O(n)$	缓存最友好，竞赛首选；扩容会使所有迭代器/指针/引用失效
array<T,N>	连续数组	$O(1)$	$O(n)$	X	X	长度编译期常量；比裸数组安全
string	连续	$O(1)$	$O(n)$	尾均摊 $O(1)$	$O(n)$	<code>+=</code> 均摊 $O(1)$ ，但 <code>s = s + t</code> 是 $O(n)$ 且拷贝；find 是暴力，别当 KMP
deque	分段连续	$O(1)$	$O(n)$	头尾均摊 $O(1)$ ，中间 $O(n)$	头尾 $O(1)$ ，中间 $O(n)$	常数比 vector 大；头尾都快的唯一线性容器
list	双向链表	X	$O(n)$	$O(1)$ （已知位置）	$O(1)$ （已知位置）	<code>splice</code> $O(1)$ （LRU 用）；缓存极差，竞赛少用
set / map	红黑树	X	$O(\log n)$	$O(\log n)$	$O(\log n)$	有序；插入不影响其他迭代器； <code>map[k]</code> 会默认插入（查存在用 <code>find / count</code> ）
multiset / multimap	红黑树	X	$O(\log n)$	$O(\log n)$	$O(\log n)$	键可重复； <code>erase(key)</code> 删全部，删一个要用迭代器
unordered_set / unordered_map	哈希桶	X	平均 $O(1)$ ，最坏 $O(n)$	平均 $O(1)$	平均 $O(1)$	无序；可被构造数据卡（CF 常见），重要场合用 map 或自定义 hash
priority_queue	二叉堆	X	只看堆顶 $O(1)$	push $O(\log n)$	pop $O(\log n)$	默认大顶堆；没有 <code>erase / clear</code> ；建堆 $O(n)$
stack	deque	X	—	push $O(1)$	pop $O(1)$	只操作栈顶
queue	deque	X	—	push $O(1)$	pop $O(1)$	只操作队头队尾

容器	底层	随机访问	查找	插入	删除	关键特点 / 坑
bitset<N>	位压缩	O(1)	—	—	—	位运算/ count() 都是 O(N/64)； N 必须是编译期常量
pair / tuple	—	—	—	—	—	自带字典序比较，sort 直接可用

3.3 逐个容器的「特点」说明

vector — 动态数组（默认选择）

- **有序？** 按插入顺序存，不自动排序
- 连续内存 → **随机访问 O(1)、缓存最友好、常数最小** → 所以"能用 vector 就用 vector"
- push_back 均摊 O(1)（扩容按倍数增长）；**没有 push_front**
- size() 和 capacity() 是两回事；知道总量时先 reserve(n) 避免反复扩容
- **扩容会让所有迭代器/指针/引用失效** → 不要在循环里存 &v[i]
- 典型用途：一切数组能干的事、图的邻接表、DP 数组

set / map — 红黑树（有序）

- **有序？** 自动排序（默认升序），**遍历出来就是有序序列**；能 lower_bound / upper_bound 找前驱后继
- 查找 / 插入 / 删除都是 O(log n)，**最坏也有保证** —— 这是它和 unordered_* 的本质区别
- set 的**键即值、不可修改**；map 的**键不可修改、值可改**。**要改键只能删了重插**
- 元素要支持 <（或传自定义比较器）
- **插入 / 删除不影响其他元素的迭代器**（除被删的那个）→ 可以放心边遍历边删
- 常数大：每次都是指针跳转、内存不连续 → 同样 O(log n)，比 vector 慢得多
- set 自动去重；multiset 允许重复键，erase(key) 会删**全部**，只删一个要用迭代器
- ⚠ map[k] **查不到会默认插入一个 k → 0**；只查存在请用 find / count
- 典型用途：动态有序集合、前驱后继、去重+排序、对顶堆求中位数

unordered_set / unordered_map — 哈希（无序）

- **无序！** 遍历顺序不可预测，**和插入顺序也无关**（rehash 之后还会变）
- 平均 O(1)，**最坏 O(n)** → 出题人可以构造数据把你卡成 O(n²)（CF 上很常见）
- **不支持 lower_bound** —— 无序容器没有"前驱/后继"的概念
- 键需要 hash + ==；自定义类型要自己写 hash 特化
- 插入可能触发 rehash → **所有迭代器失效**
- 典型用途：只做"存在性 / 计数"，完全不在乎顺序时

priority_queue — 堆（不是有序容器）

- **只保证堆顶是最值，内部完全无序**，也不能查任意元素
- 没有 erase、没有 clear、没有迭代器 → 清空只能 while(!pq.empty()) pq.pop();
- **默认大顶堆**；小顶堆写法：priority_queue<int, vector<int>, greater<int>>
- 自定义类型要写 operator<，而且**方向和直觉相反**（< 定义为"优先级低"）
- 建堆 O(n)；push / pop O(log n)
- 和 set 的分工：**只要最值 → 堆（常数小）**；**要任意查询/删中间 → set**
- 典型用途：贪心取最大、Dijkstra、k 路归并

deque — 双端队列

- 头尾都能均摊 O(1) 插入删除，**且支持随机访问** → 唯一"两头都快"的线性容器
- 分段连续 → 常数比 vector 大；**头尾插入不失效引用**（但失效迭代器）
- 典型用途：单调队列（但竞赛里手写数组版更快）

list — 双向链表

- 任意位置 $O(1)$ 插入/删除（前提是已经持有迭代器）；**不支持随机访问**
- 插入/删除不影响其他元素；splice 是 $O(1)$ （整段搬家）
- **缓存极差**（每次跳指针）→ 竞赛基本不用，除非需要 splice
- 典型用途：LRU 缓存

stack / queue — 容器适配器

- **不是容器**，是套在 deque 外面的适配器，只暴露极少接口
- queue 没有 clear()；想遍历 stack 只能边弹边存
- 典型用途：DFS 手写栈、BFS 队列

string

- 和 vector<char> 类似，但 += 均摊 $O(1)$ ，而 $s = s + t$ 是 $O(len)$ 且会拷贝
- find 是**暴力匹配 $O(n \cdot m)$** ，别当 KMP 用
- substr(pos, len) 会拷贝出新串， $O(len)$

bitset<N>

- **N 必须是编译期常量**；位压缩存储 → 空间约为 bool 数组的 1/8
- count()、& | ^ ~ << >> 全部是 $O(N/64)$
- 典型用途： $O(n^3/64)$ 优化传递闭包、位运算 DP、状态压缩

array<T,N> / pair / tuple

- array：长度编译期常量 + 越界检查，相当于"更安全的裸数组"
- pair / tuple：**自带字典序比较**（先比 first），sort 直接可用 → 需要按多个关键字排序时**优先用 pair**，比手写结构体省事

3.4 选型决策表（先问需求，再选容器）

你的需求	用	别用	原因
按顺序存 + 频繁下标访问	vector	list / set	连续内存最快
两头都要插入删除	deque	vector（没有 push_front）	只有 deque 两头都均摊 $O(1)$
只在一端插入删除	stack / queue	手写 deque	接口更清晰、意图更明确
动态有序 + 找前驱后继	set / map	unordered_*（ 不能 lower_bound）	哈希无序
只要存在性 / 计数	unordered_*	set / map（慢一个 log）	平均 $O(1)$
只要最值	priority_queue	set（杀鸡用牛刀）	堆常数小得多
要删/查中间任意元素	set	priority_queue（ 没有 erase）	堆只暴露堆顶
按多个关键字排序	pair / tuple	手写结构体	自带字典序比较
位运算 / 状态压缩	bitset<N>	vector<bool>（特化，坑多）	位压缩 + $O(N/64)$
$O(1)$ 插删且已持有位置	list	vector（中间是 $O(n)$ ）	链表节点 $O(1)$

四个核心容器的分工，一句话各一条

- vector —— **默认**。没有特殊需求就用它。
- set / map —— ****要**有序 + 任意查询*****时用，代价是 log 和较大的常数。
- unordered_* —— **只要**快速查在不在****、不关心顺序时用；但要记住它最坏 $O(n)$ 。

- `priority_queue` —— 只要最值、不要全序时用，比 `set` 快得多。

3.5 三个必须记住的"经典坑"

1. `set / map` 的 `lower_bound` 要用成员函数：`s.lower_bound(x)` 是 $O(\log n)$ ；`std::lower_bound(s.begin(), s.end(), x)` 是 $O(n)$ （红黑树不支持随机访问）。
2. `vector` 扩容后旧迭代器/指针/引用全部失效，但 `deque` 头尾插入不失效引用，`list / set / map` 插入不影响其他迭代器。
3. `unordered_map` 的最坏复杂度是 $O(n)$ 。被卡了换 `map`（ $O(\log n)$ 但稳定），或自定义随机 `hash`。

四、常用 STL 算法 / 工具：复杂度 + 用法

工具	复杂度	用法 / 注意
<code>sort</code>	$O(n \log n)$	比较函数必须是 严格弱序 （ <code>return a < b</code> ），写 <code><=</code> 会 UB / RE
<code>stable_sort</code>	$O(n \log^2 n)$ （空间够则 $O(n \log n)$ ）	需要保持相等元素原顺序时
<code>nth_element</code>	平均 $O(n)$	求第 k 小： <code>nth_element(a, a+k, a+n)</code> ，左边都比它小
<code>lower_bound / upper_bound</code>	$O(\log n)$ 次比较	必须有序 ； <code>lower</code> 找第一个 $\geq$ ， <code>upper</code> 找第一个 $>$
<code>unique</code>	$O(n)$	只能去相邻重复 → 先 <code>sort</code> ；返回新末尾，配合 <code>erase</code>
<code>reverse</code>	$O(n)$	反转区间
<code>next_permutation</code>	枚举全部是 $O(n \cdot n!)$	先 <code>sort</code> 得到最小排列
<code>accumulate</code>	$O(n)$	初始值写 0LL ，写 <code>0</code> 会按 <code>int</code> 累加溢出
<code>max_element / min_element</code>	$O(n)$	返回迭代器
<code>merge</code>	$O(n+m)$	两个有序区间合并
<code>partial_sum</code>	$O(n)$	前缀和（ <code>plus</code> ）/ 前缀积
<code>fill / iota</code>	$O(n)$	<code>iota</code> 填 <code>0,1,2,...</code>
<code>gcd / lcm</code> （C++17）	$O(\log n)$	<code>std::gcd(a,b)</code>
<code>__builtin_popcount</code>	$O(1)$	数 1 的个数； <code>popcountll</code> 用 <code>long long</code>
<code>__builtin_clz / ctz</code>	$O(1)$	前导零 / 末尾零个数（求最高位/最低位）
<code>max / min / swap / abs</code>	$O(1)$	注意 <code>abs</code> 对 <code>long long</code> 要用 <code>llabs</code> 或 <code>std::abs</code> 重载

用法示例（可直接抄）

排序：多关键字 + 严格弱序

```
vector<pair<int,int>> v;
sort(v.begin(), v.end()); // pair 自带字典序：先 first 再 second，省事

struct Node { int a, b; };
bool cmp(const Node &x, const Node &y) {
    if (x.a != y.a) return x.a < y.a; // 第一关键字升序
    return x.b > y.b; // 第二关键字降序
}
sort(arr + 1, arr + n + 1, cmp); // ⚠ [first, last): 要写 n+1
stable_sort(arr, arr + n, cmp); // 相等元素保持原相对顺序
```

- ⚠ 易错：边界（`sort(a+1, a+n)` 漏掉 `a[n]`）· 逻辑（比较函数里写 `<=` → 破坏严格弱序 → UB/RE）

lower_bound / upper_bound：有序容器上的前驱后继

```
int pos = lower_bound(a, a + n, x) - a; // 第一个 >= x（不存在 = n）
int cnt = upper_bound(a, a+n, x) - lower_bound(a, a+n, x); // x 的出现次数

auto it = s.lower_bound(x); // 第一个 >= x ← 后继
auto it2 = s.upper_bound(x); // 第一个 > x
auto pre = (it == s.begin()) ? s.end() : prev(it); // 严格小于 x 的最大值 = 前驱
```

- 要点：前驱 = `prev(lower_bound(x))`，后继 = `lower_bound(x)`（`x` 本身不算）。
- ⚠ 易错：逻辑（在 `set / map` 上用 `std::lower_bound` → $O(n)$ ）· 边界（解引用 `end()`）

第 k 小 / 第 k 大

```
nth_element(a, a + k, a + n); // 0-based: a[k] 是第 k+1 小 ← 坑在这
nth_element(a, a + n - k, a + n); // 第 k 大

// 动态维护（边插边问）——小顶堆里只留 k 个最大的
priority_queue<int, vector<int>, greater<int>> q;
for (int x : a) { q.push(x); if ((int)q.size() > k) q.pop(); }
cout << q.top(); // 第 k 大
```

- 要点：`nth_element` 是 **0-based 重灾区**：求“第 k 小”要传 `a + k - 1`。
- ⚠ 易错：边界（下标差 1）· 逻辑（动态维护时 `size() > k` 写成 `>= k`）

accumulate / 位运算内建函数

```
long long sum = accumulate(a, a + n, 0LL); // ⚠ 初始值必须写 0LL
int mx = *max_element(a, a + n); // 返回迭代器，要解引用
partial_sum(a, a + n, s); // 前缀和（默认 plus）
iota(fa, fa + n + 1, 0); // 依次填 0,1,2,...

int cnt = __builtin_popcount(x); // 1 的个数
int hi = 31 - __builtin_clz(x); // 最高位下标（要求 x > 0）
int lo = __builtin_ctz(x); // 最低位下标（要求 x > 0）
```

- ⚠ 易错：类型（`accumulate` 初始值写 `0` → 按 `int` 溢出）· 边界（`clz / ctz` 对 0 是未定义行为）

bitset : $O(n^3/64)$ 传递闭包

```
const int M = 2000;
bitset<M> bs[M];
bs[u][v] = 1; // 有边  $u \rightarrow v$ 
for (int k = 0; k < n; k++) // 三重循环压成两重
    for (int i = 0; i < n; i++)
        if (bs[i][k] & bs[k][j]) bs[i][j] |= bs[k][j];
cout << bs[i].count(); //  $O(M/64)$ 
```

- 要点: M 必须是编译期常量; count()、&、|、^、~、<<、>> 全部 $O(N/64)$ 。
- ⚠ 易错: 逻辑 (M 用运行时变量 → 编译不过)· 边界 (M 开小 → 越界)

unordered_map 防卡 (CF 上真会被卡)

```
struct custom_hash {
    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15ULL;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
        x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
        return x ^ (x >> 31);
    }
    size_t operator()(uint64_t x) const {
        static const uint64_t FIXED_RANDOM =
            chrono::steady_clock::now().time_since_epoch().count();
        return splitmix64(x + FIXED_RANDOM);
    }
};
unordered_map<long long, int, custom_hash> mp;
```

- 要点: unordered_map 默认 hash 对整数近似"取固定质数模" → 质数固定 ⇒ 可以构造出全冲突的数据。加随机种子就卡不掉了。
- 备选: 直接换 map, 稳定 $O(\log n)$, 代价是常数。

next_permutation 枚举排列

```
sort(a, a + n); // ⚠ 必须先排序, 否则会漏解
do {
    // 处理当前排列
} while (next_permutation(a, a + n));
```

- 要点: 枚举全部是 $O(n \cdot n!)$; $n = 10$ 约 $3.6e6$, $n = 11$ 就 $4e7$ 了 —— 蓝桥杯填空题常用。

浮点与输出格式

```
cout << fixed << setprecision(2) << ans << "\n"; // 保留 2 位小数
cout << setw(5) << x; // 右对齐占 5 格 (只对紧邻的下一个元素有效!)
cout << left << setw(5) << x; // 左对齐
cout << setw(2) << setfill('0') << hh << ":"; // 前导零
```

- ⚠ 易错: 逻辑 (setw 只影响紧接着的一个输出, 第二个元素必须重写 setw)

五、复杂度预算表（看数据量反推解法）

n 上界	只能接受	常见做法
≤ 10	$O(n!)$	全排列枚举、DFS 回溯
≤ 20	$O(2^n \cdot n)$	状压 dp、枚举子集
≤ 40	$O(2^{(n/2)})$	meet in the middle
≤ 100	$O(n^3)$	Floyd、区间 dp、矩阵乘法
≤ 500	$O(n^3 / 64)$	bitset 优化、 $O(n^2 \log n)$
≤ 5000	$O(n^2)$	二维 dp、LCS、编辑距离
≤ 10^5	$O(n \log n)$ 、 $O(n\sqrt{n})$	排序 + 双指针、树状数组、线段树、莫队
≤ 10^6	$O(n)$ 或 $O(n \log n)$ 且常数小	前缀和、双指针、单调队列、线性筛
≤ 10^9	$O(\log n)$	快速幂、二分答案、数位 dp
≤ 10^{18}	$O(\log n)$ 、 $O(\log^2 n)$	矩阵快速幂、扩展欧几里得

参考速度：1 秒约 10^8 次简单运算。所以 $n = 10^5$ 时 $O(n^2)$ (10^{10}) 必超时， $O(n \log n)$ (1.7×10^6) 非常轻松。
用法：拿到题先看数据量 → 在这个表里定位"只能接受"那一行 → 反推该用什么复杂度 → 再去「按问题查解法」里挑结构。这就是讲解 087「根据数据量猜解法」的全部内容。

六、跨专题常见坑（持续追加）

坑	对应错因标签
数组开的是"元素个数"，但实际要开"节点数 × 字符集"（字典树、AC 自动机）	边界
多测 memset 整张大表 → 超时（改成只清用过的部分）	时间
sort(a+1, a+n) —— STL 区间是 [first, last)，漏了 a[n]	边界
比较函数写 <= → 严格弱序被破坏 → RE / UB	逻辑
accumulate 初始值写 0 而不是 0LL	类型
答案可能超过 int（方案数、路径条数、异或和）	类型
同一个量在两个地方都赋值，第二处静默覆盖第一处	逻辑
输出用 endl 而不是 '\n'（大量输出时 TLE）	时间
map[k] / unordered_map[k] 读操作也会默认插入	逻辑

完整版：按 note.md 的错因标签归类

标签取自 templates/note.md。每条后面附**自查方法**——目的是让你在提交前自己抓到，而不是等 WA 之后才归类。

边界 — 数组越界、n=1 特判、负数/零处理

- 数组按"元素个数"开，实际要按**节点数 × 字符集**开（字典树 / AC 自动机 / 01-Trie）
- sort(a+1, a+n) —— STL 是 [first, last)，漏了 a[n]

- 差分数组用 `d[r+1]` → 必须开 `n+2`
- 下标从 1 开始的结构，循环写成 `i >= 0` (应为 `i > 0`)
- `n = 1`、`k = 1`、`k = n`、全相同、全 0
- 无解情况：题面要求的输出是什么？
- `prev(begin())`、解引用 `end()` 迭代器
- `__builtin_clz(0)` / `ctz(0)` 是未定义行为

自查：把 `n` 取最小值、`k` 分别取 1 和 `n`，手跑一遍。

类型 — int 溢出、忘记 long long、浮点精度

- 前缀和 / dp / 方案数 / 路径条数 / 异或和 忘记 `long long`
- `accumulate(..., 0)` 写成 `0` 而不是 `0LL`
- **乘法先溢出再赋值：**要写 `1LL * a * b`，不能靠左边的 `long long` 变量救
- `abs` 用在 `long long` 上 (要用 `llabs` 或 `std::abs` 重载)
- 浮点用 `==` 比较

自查：把最大值代进每个中间表达式，心算会不会超 2^{31} 。

初始化 — 多测没清空、默认值不对、INF 设置错误

- 多测没清空：并查集 `fa[i]=i`、dp 数组、答案变量、图的邻接表
- 多测 `memset` 整张大表 → 改成只清 `0..idx` 用过的部分
- 求 `min` 的 dp 忘了初始化成 `INF`；求 `max` 初始化成 `-INF`
- 背包计数忘了 `dp[0] = 1`
- 建树/建表忘了先 `idx = 0`

自查：假装自己是评测机，连跑 3 组数据，看第 2、3 组会不会被第 1 组污染。

逻辑 — 算法思路错误、条件漏判、推导不严密

- 比较函数写 `<=` → 严格弱序被破坏
- **同一个量在两处赋值，第二处静默覆盖第一处** (你的高频错因)
- 双指针收缩用 `if` 而不是 `while`
- 二分里 `hi = mid - 1` 与 `hi = mid` 混用 → 漏解 / 死循环
- 在 `set` / `map` 上用 `std::lower_bound` ($O(n)$)
- `map[k]` / `unordered_map[k]` 读操作也会默认插入
- 单调栈/队列：队首过期判断、入队时机
- 拓扑排序忘了判环 (入队数 $< n$)
- 字典树：`p[]` (经过=前缀数) 和 `e[]` (结尾=整串数) 混用
- 01-Trie：贪心时只走 `path` 不走 `path ^ 1`

自查：写完用一句话说明清每个状态/变量的含义。说不清的，问题就在这里。

实现 — 思路对但代码写错、变量名混淆、循环边界

- 循环边界写反 (`<=` 与 `<`)
- **运算符优先级：**`a & 1 == 0` 实际是 `a & (1 == 0)`
- **移位溢出：**`1 << k` 在 $k \geq 31$ 时是 UB，要用 `1LL << k`
- 线段树忘记 `pushup` / `pushdown`
- 并查集路径压缩写错 (`fa[x] = fa[fa[x]]`)
- 手写队列/栈的 `head` / `tail` 初值

自查：把这些结构当“模板”背一遍，能背出即说明你写的是对的。

时间 — 超时、复杂度假了

- 多测 memset 大数组
- unordered_map 被构造数据卡
- endl 代替 '\n'
- vector<vector<int>> 做二维 dp 比静态数组慢很多
- 递归深度过大（栈溢出 / 卡常）
- 复杂度对了但常数大（map 换掉 unordered_map、list 换掉 vector）
- 忘了 ios::sync_with_stdio(false); cin.tie(nullptr);

自查： n 取最大，算 最坏操作次数 $\div 1e8$ ，超过 1 就得重新想。

读题 — 漏看条件、理解偏差

- 漏看"答案对 $10^9 + 7$ 取模"
- 漏看多测（ T 组数据）
- 漏看无解时的输出（-1 / 空行 / 长度 0 的数组）
- 漏看下标从 0 还是从 1 开始
- 答案要求"字典序最小"还是"任意一组"
- 数据范围的量级与单位（ 10^5 还是 10^9 ；求和还是求积）

自查：把题面的**加粗 / 数字 / 输出格式**三处各读第二遍。

提交前 60 秒自查清单

1. 数组开够了吗？（最大节点数 / 最大下标 + 2）
2. 该 long long 的地方都是 long long 吗？中间乘法会不会先溢出？
3. 多测清空了吗？（并查集、dp、idx、答案变量）
4. 极端数据手跑过吗？（ $n=1$ 、全相同、全 0、最大范围）
5. 输出格式对吗？（换行 / 取模 / 无解输出）
6. 复杂度和 n 对得上吗？（回查第五节表）
7. 有没有**同一个量写了两遍**？

七、待补清单（本文件的下一步）

- ☒ 每个数据结构的「一句话用法」展开成**可复制的最小代码块** → 已完成，见第二节「最小可复制代码块」
- ☒ STL 算法的**用法示例** → 已完成，见第四节「用法示例」
- ☒ 「常见坑」按错因标签归并成完整版 + 提交前自查清单 → 已完成，见第六节
- ☐ 图论部分补：网络流、二分图匹配、差分约束
- ☐ 字符串部分补：Z 函数、后缀数组、AC 自动机
- ☐ 数学部分补：莫比乌斯、CRT、高斯消元、组合计数
- ☐ 「按问题查解法」补上**每个结构的适用边界**（什么情况下会挂）
- ☐ STL 继续补：mt19937 随机数、pbds 平衡树（tree 求第 k 小 / 排名）